Progress Report - 1

on

Subscription Leak Detector

Submitted in the partial fulfilment of the Degree of Bachelor of Technology
(Information Technology)

Submitted by

Shubh Shekhar Jha (Enrollment No: 02096303123)
Anchal Paswan (Enrollment No: 04496303123)
Karma (Enrollment No: 04996303122)

Under the supervision of

Dr. Manoj Malik

Department of Information Technology
Maharaja Surajmal Institute of Technology
Janakpuri, New Delhi.
2023-2027

Progress Report- 1

Mentor Feedback Implementation

Subscription Leak Detector — Synopsis Revision Status

Team – Shubh Shekhar , Anchal, Karma

Mentor – Dr. Manoj Malik

Summary

Of the 13 points raised, 11 are fully addressed in the revised synopsis, and 2 are partially addressed with concrete next steps identified in this report. No point has been left untouched — every item has at least a documented response, and the majority have new formal sections, formulas, tables, or diagrams added directly into the synopsis document.

Overall completion: ~90%

Point-by-Point Status

#	Mentor's Point	Status	Location in Synopsis
1	Reduce scope; define core vs. optional modules	✓ Done	*Written response provided; recommend formalizing as a dedicated "Scope" page (see below)*
2	Clearly define the AI/ML component (I–VII)	✓ Done	*Page 7 — "AI/ML Problem Definition"*
3	Specify dataset and ground-truth labelling strategy	✓ Done	*Page 8*
4	Distinguish recurring transactions from actual subscriptions	✓ Done	*Page 8*
5	Replace "risk score" with Subscription Confidence/Review Score	✓ Done	*Pages 2, 3, 6, 10 (renamed throughout)*
6	Do not claim bank data determines actual usage	✓ Done	*Page 8 — "Terminology and Score Interpretation"*
7	Define mathematical/statistical method for recurrence detection	✓ Done	*Page 9 — "Recurrence Detection Method"*
8	Define price-increase detection criteria	✓ Done	*Page 9 (single-cycle check) + Page 11 (full price-creep methodology)*
9	Feedback as collection mechanism now; retraining as advanced feature	✓ Done	*Page 9 — "Feedback Mechanism: Collection versus Retraining"*
10	Add clear experimental evaluation and baseline comparison	⚠ Partial	*Page 12 — table structure exists; two rows still need real numbers (see below)*
11	"Risk Score" is not properly defined	✓ Done	*Page 10 — "Subscription Confidence Score — Formal Definition" (full formula + weight table)*

#	Mentor's Point	Status	Location in Synopsis
12	Price-creep detection needs a clear methodology	✓ Done	*Page 11 — "Price-Creep Detection — Full Methodology" (step-by-step + worked example)*
13	System Architecture (with diagram)	✓ Done	*Page 12 — full architecture diagram + explanation*

Questions And Answers :-

1 Reduce the scope and define core versus optional modules.
2 Clearly define the AI/ML component. I. What exactly is the ML problem? II. What is the input feature vector? III. What is the target/output? IV. Is the task classification, clustering, anomaly detection, or ranking? V. Which algorithms will be compared? VI. How will the model be trained? VII. Where will labelled training data come from?
3 Specify the dataset and ground-truth labelling strategy.
4 Distinguish recurring transactions from actual subscriptions.
5 Replace vague "risk score" terminology with a properly defined Subscription Confidence/Review Score.
6 Do not claim that bank data can determine whether a subscription is actually unused.
7 Define the mathematical/statistical method for recurrence detection.
8 Define the price-increase detection criteria.
9 Implement user feedback as a feedback collection mechanism; periodic retraining can be an advanced feature.
10 Add a clear experimental evaluation and baseline comparison.
11 The "Risk Score" is not properly defined.
12 Price creep detection needs a clear methodology 13 System Architecture
These are the questions that my mentor gave to me after showing the synopsis report to him and these are the changes have to be made and also want to tell how much is done and how much is left to do in this report.

Answers :
1. Reduce Scope — Core vs Optional Modules
What he means: Right now everything (chart, simulator, feedback, auth) is presented as equally important. He wants you to clearly separate "the thing that must work for this to be a valid ML project" from "nice extra features."
How to answer it:
Core Modules (the actual ML contribution):

- Merchant entity resolution (text cleaning + fuzzy matching)
- Recurrence detection (statistical classification)
- Subscription Confidence Scoring

Supporting Modules (needed to demo it, but not the ML contribution itself):

- Django backend, MySQL storage, REST API
- Frontend dashboard, authentication

Optional/Stretch Modules (explicitly label these as such):

- Price-history chart
- Savings simulator
- Feedback collection UI
- Multi-user support

2. I. What exactly is the ML problem?
There are actually two distinct sub-problems — say this explicitly, it shows clarity:

1. Entity resolution (merchant name matching) — an unsupervised similarity/clustering problem

2. Recurrence classification — a binary classification problem: is this cluster of transactions a recurring subscription (yes/no)?

II. What is the input feature vector?

For the classification stage, per merchant cluster:

[transaction_count, gap_mean_days, gap_std_days,
amount_mean, amount_std (price_stability),
price_increase_pct, days_since_last_charge]

Name these exact 7 features when asked — this is your feature vector.

III. What is the target/output?

Two outputs, at two stages:

- Stage 1 output: is_recurring — binary (0/1)
- Stage 2 output: subscription_confidence_score — continuous, 0 to 1

IV. Classification, clustering, anomaly detection, or ranking?

Be precise — it's three different task types across the pipeline:

- Merchant matching = clustering (unsupervised)
- Recurrence detection = binary classification
- Price-creep detection = anomaly/change-point detection (statistical, on the price sequence)

V. Which algorithms will be compared?

This is currently your weakest area — be honest, then propose the fix:

- Current state: rule-based heuristic (thresholds on gap std dev, cycle length)
- What to compare it against: Logistic Regression and Random Forest, trained on your labeled synthetic data, using the same 7 features
- ″Our baseline is a rule-based heuristic. We propose comparing it against Logistic Regression and Random Forest classifiers trained on the same feature vector, to determine whether a trained model outperforms the hand-tuned rules."

VI. How will the model be trained?

1. Generate synthetic labeled dataset (already built)
2. Extract the 7 features per merchant cluster
3. Split into train/test sets
4. Train Logistic Regression / Random Forest using scikit-learn
5. Evaluate on held-out test data (precision, recall, F1)

VII. Where will labelled training data come from?

Your synthetic data generator — ″Since real bank statements aren't accessible due to privacy constraints, we built a synthetic data generator that creates realistic transaction sequences with known recurring merchants, known frequency, and known price-increase patterns baked in — giving us verified ground-truth labels to train and evaluate against."

3. Specify Dataset and Ground-Truth Labelling Strategy

"Our dataset is synthetically generated using the Faker library. It contains 6 pre-defined recurring merchants with known frequency (weekly/monthly/annual) and scheduled price increases, combined with ~80 random one-off transactions as noise. Each transaction is generated with a ground-truth label (is_recurring: True/False, true_merchant_name) that is withheld from the detection pipeline but used afterward for evaluation. This gives us a verifiable, reproducible test set despite not having access to real user bank data."

4. We're upfront that this is a real distinction — rent and EMIs are recurring too, but they're not subscriptions. Technically, our system detects recurring merchant charges. We narrow that down to subscription-likely ones using category heuristics — entertainment, SaaS, fitness, and cloud-storage merchants are prioritized, while things like rent or utility categories are deprioritized or excluded. We treat this as an explicit design assumption, not a fully solved problem."

5. "'Risk score' implied we were predicting financial danger, which overclaims what the system actually does. We renamed it to Subscription Confidence Score — a measure of how strongly a

recurring pattern resembles a subscription worth reviewing, based only on payment behavior, not a judgment about the user's finances."

6 "No, and we're explicit about that limitation. Bank data only shows that a payment happened — it has no information about whether the service was actually used. Our system flags recurring charges as review-worthy based on payment patterns like price stability and recency, but the final judgment of usage always rests with the user, through the feedback step."

7. "For a merchant cluster's sorted transaction dates, we compute the gaps between consecutive charges, then take the mean and standard deviation of those gaps. We classify a cluster as recurring if three conditions hold: at least 3 transactions exist, the gap standard deviation is 5 days or less, and the mean gap falls within 5 days of a known billing cycle — 7, 30, or 365 days."

8 "We actually have two levels. The quick single-cycle check splits a cluster's transactions into an early half and late half, compares average amounts, and flags anything over a 10% increase. But for genuine price creep — sustained increases over multiple cycles — we use a more rigorous method: computing every consecutive percentage change, counting how many exceed 2%, and flagging creep only if the cumulative growth exceeds 10%, at least one step-increase occurred, and no single step alone exceeds 50% — which would indicate a one-off plan change rather than gradual creep."

9. "Right now, it's feedback-informed heuristic adjustment, not model retraining. If a user has previously marked a merchant 'cancel,' we boost its confidence score by a fixed amount on the next upload; if they confirmed continued use, we lower it. Full periodic retraining — refitting a Logistic Regression or Random Forest using accumulated feedback as new labels — is proposed as an advanced feature once we've collected enough feedback volume."

10. "We compare our proposed statistical method against a naive baseline — classifying any merchant appearing twice or more as recurring, with no consistency checks. Our method achieved 100% precision and about 83% recall on our synthetic test set, with zero false positives; the one miss was an annual subscription with only two data points in our observation window, which our own minimum-transaction-count rule correctly excludes. We're currently finishing two things: computing the naive baseline's actual numbers, and running the Logistic Regression / Random Forest comparison — both are scoped and should take a few hours total."

11. "It's a weighted sum of three binary signals plus a baseline, capped at 1.0: C equals the baseline, 0.20, plus 0.35 if the price increased more than 10%, plus 0.25 if the price has been very stable, plus 0.20 if there's been no recent charge. So the base score always sits between 0.20 and 1.00 for anything already classified as recurring. On top of that, we apply a separate, clearly-labeled feedback adjustment — plus 0.15 if the user previously said 'cancel,' minus 0.20 if they confirmed continued use."

12. "We sort a cluster's amounts chronologically, then compute the percentage change between every consecutive pair. Any step over 2% counts as an 'increase event.' We then compute the cumulative growth from the very first to the very last charge. We flag genuine price creep only if that cumulative growth exceeds 10%, at least one increase event occurred, and no single step by itself exceeded 50% — the last condition specifically rules out a one-time plan upgrade being mistaken for gradual creep. For example, a subscription going ₹199 → ₹249 → ₹299 shows two increase events and 50.3% cumulative growth, correctly flagged as price creep."

13. "The user uploads a CSV through the frontend to a CSRF-protected, session-authenticated Django REST API. That data flows through our ML pipeline — cleaning, clustering, recurrence detection, and confidence scoring, in that order. Results are saved to MySQL, scoped to that logged-in user, across Transactions, Merchants, Subscriptions, and Price History tables. The dashboard reads from there to

show subscription cards, the price chart, and the savings simulator. When the user submits feedback, it's stored and used to adjust that merchant's confidence score the next time they upload — which is the loop shown on the diagram."

What Was Actually Done (Detail)

- A formal 7-dimensional feature vector, explicitly named
- The exact weighted formula for the Subscription Confidence Score: $C = \min(1.0, b + w_1 \cdot I_1 + w_2 \cdot I_2 + w_3 \cdot I_3)$, with every weight and condition tabulated
- A step-by-step statistical method for recurrence detection, using gap mean/std-dev and billing-cycle proximity
- A separate, more rigorous multi-cycle price-creep methodology with a full worked numerical example (₹199 → ₹249 → ₹299, cumulative growth calculation shown)
- An explicit statement that bank data cannot indicate actual usage — the score is described only as a payment-pattern confidence measure
- A real system architecture diagram showing the complete data flow, including the feedback loop
- Clear separation of “feedback collection + heuristic adjustment” (implemented now) from “periodic model retraining” (explicitly labeled future work, not implemented)

Terminology

“Risk Score” has been replaced with “Subscription Confidence Score” consistently across the abstract, objectives, research gap analysis, and the new formal-definition page — not just in one place.

Scope

Addressed via a written breakdown separating three tiers:

- Core modules: merchant entity resolution, recurrence detection, confidence scoring
- Supporting modules: Django backend, MySQL storage, REST API
- Optional/stretch modules: price-history chart, savings simulator, feedback UI, multi-user support

This exists as a clear explanation ready to present, but has not yet been inserted as its own dedicated page in the synopsis document. Recommended next step: add a one-page “Project Scope” section near the Objectives page, formally listing these three tiers.

What's Left To Do :-

The evaluation table on Page 12 currently looks like this:

Method	Precision	Recall	F1
Naive baseline (merchant appears ≥ 2 times)	Low (qualitative)	High (qualitative)	Low (qualitative)
Proposed statistical method	**100%**	**83%**	**~0.90**
Logistic Regression / Random Forest	To be evaluated	To be evaluated	To be evaluated

Two rows still need real numbers instead of qualitative placeholders:

1. Naive Baseline — Needs an Actual Computed Number

This requires re-running the existing synthetic dataset through a simple rule ("any merchant appearing ≥ 2 times = recurring") and computing real precision/recall against the same ground truth already used for the proposed method. Estimated effort: ~20–30 minutes, since the dataset and evaluation code already exist — this is just one more pass with a simpler rule.

2. Logistic Regression / Random Forest Comparison — Not Yet Implemented

This requires: extracting the 7-feature vector into a training set, splitting train/test, training both classifiers with Scikit-learn, and evaluating on the held-out set. Estimated effort: 1–2 hours, since the feature-extraction code already exists in the pipeline; only the training/evaluation script is new work.

Recommendation for the Meeting

Present the current state honestly — the framework and methodology for this comparison is fully defined and one row is real, verified data; the remaining two numeric gaps are scoped with clear, short effort estimates rather than being open-ended unknowns. This is a stronger position than pretending the numbers already exist.

Suggested One-Line Summary for the Meeting

"Of the 13 points raised, 11 are fully written into the revised synopsis with formulas, tables, and diagrams. The remaining item — completing the baseline comparison table with real naive-baseline and classifier numbers — is scoped and estimated at 2–3 hours of additional work, which we can complete this week."